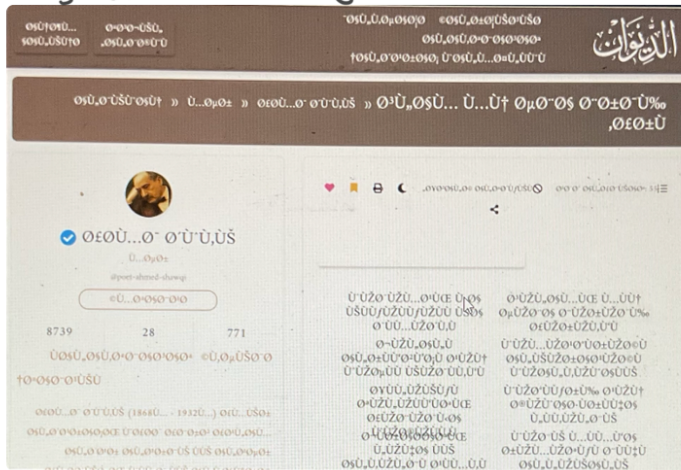


Data Encoding :- (Learn how computer encodes characters, from ASCII to unicode & UTF.)

Task 1 :- Introduction

In previous room, you learned how numbers are represented in computer as binary digits (bits) and how these bits can be grouped into hexadecimal digits; we also explored how to express colors using numbers. If everything stores as numbers, how do we convert them to letters, punctuation and emojis? In this room, you will learn that characters are just numbers with agreed meanings. The Agreement is called an **Encoding**. Have you ever opened a document, visited a web page or downloaded movies subtitle files appear as weird gibberish? One reason user who saved file and user who opened it are using different Encodings.



Representation is idea that data lives as bits and numbers in memory. At same time, encoding is specific, agreed upon mapping b/w numbers and meanings, such as which number corresponds to character "A". In text, each character like 'a', '3' or '!', is assigned to numeric code, so strings become sequence of numbers that computer stores and moves around just like any other data.

Learning Objectives :-

upon completion of this room, you will learn about :-

* ASCII * unicode * UTF-8, UTF-16 & UTF-32 * How emoji is encoded * And what causes weird gibberish characters.

Task 2 :- ASCII

We already learned digital computers only understand 0 and 1. Ex:- how can we save the text

"Tru Hack Me" in file? what will such a file contain?

* To be able to answer, we need to understand what bits represent T, what bits represent R, what bits represent Y.

Let's say T with following stream of bits 01010100. Then all computers should store T same way.

This approach required a standard that computer manufacturers, designers and programmers agree to adhere to. One of the earliest standard for English letters was ASCII.

ASCII stands for American standard code for information interchange, its early character encoding from 1963 that uses numbers 0-127 to represent English letters, digits, punctuation and some control characters. ASCII acts as small bilingual dictionary b/w text and numeric codes. Consider following sample from original ASCII table.

Decimal	Hexadecimal	Binary	Symbol	Description
48	30	00110000	0	Zero
65	41	01000001	A	Uppercase A
88	58	01011000	X	Uppercase X
89	59	01011001	Y	Uppercase Y
97	61	01100001	a	Lowercase A
98	62	01100010	b	Lowercase B
99	63	01100011	c	Lowercase C

First letters are in order. If you have hexadecimal or decimal number for 'b', you can figure out decimal number of a, c, and later lowercase letters. Ex:- a, b, c are assigned to 61, 62 and 63 respectively. Same for uppercase letters A to Z and digits 0 to 9. When your ASCII encoding a computer system reads 41 in file and displays A on screen, it reads B and so on.

Secondly, each character has its own ASCII, Example E is represented by hexadecimal number 5B.

"ToyHack Me" in ASCII

Let's say you open a text file, write "ToyHack Me" and save it as file.txt. How will file look on bit level? Let's find out.

If you are interested in seeing storage on disk, bit by bit, assuming that it is using ASCII, you will see something similar to following: 01010100 01110010 01110011 01100011 01100011.

Obviously, that's not readable by human being. There are exact binary representation of all letters

in "TryHackMe" followed by a new line. TryHackMe \n; the \n is new line that you get when you hit enter key.
Because reading binary numbers is cumbersome and error prone for us, we prefer to use hexadecimal digits.

Our file looks like this in hexadecimal: 54 72 79 48 61 63 66 4d 65 0a

And if you want to look up decimal representation, it would be as follows: 124 162 171 110 141 143 153 115 145 012,
we more commonly use hexadecimal digits.

European languages :-

ASCII provided a way to encode english alphabet; however we need an encoding to support other European languages such as spanish (ñ, ñ), Germany (ß, ü), polish (ł, ł), Czech (č, ř), and Romanian (ș, ț), to name a few. ASCII uses 7 bits, and with an eight bit, we get 128 more characters to cover. Additional 128 characters are not enough to cover all letters of European languages.

* ISO-8859-1 (Latin 1) :- covered western European languages like German, French, Spanish, Italian, Portuguese, Catalan, and Nordic languages.

* ISO-8859-2 (Latin 2) :- Support central / Eastern European languages like Polish, Czech, Hungarian, Croatian, Romanian and Slovak.

Quiz :-

1) What is ASCII code in decimal for character @?

A) 64

2) What is character that has ASCII code of 35 in decimal?

A) #

Unicode :-

Unicode is universal character encoding standard. It assigns unique code points to characters from all modern and historical writing systems worldwide. Unicode supports the interchange, processing, and display of text in diverse languages.

Unicode is character set standard that assigns a unique number to every character from all

languages example
U+0041 = latin "A" | U+03A9 = Greek "Ω" | U+3042 = Japanese language "あ"

* Unicode 17 is currently latest version of unicode standards. It defines to close of 157 thousand characters, almost 4,000 of them are emoji sequences.

UTF-8, UTF-16, UTF-32 :-

UTF-8, UTF-16, and UTF-32. UTF-8 is very common on modern web.

It encodes Unicode points into 1 to 4 bytes dynamically. ASCII characters (U+0000 to U+007F).

* UTF-16 takes a different path; it uses either 2 or 4 bytes per character.

Ex :- A letter is encoded as U+0041, while emoji  needs two and is encoded as U+D83D U+DD25.

Finally UTF-32 is simplest but also most wasteful, every unicode point uses 4 bytes.

Example, A is encoded as U+00000041 and  is encoded as U+0001F525.